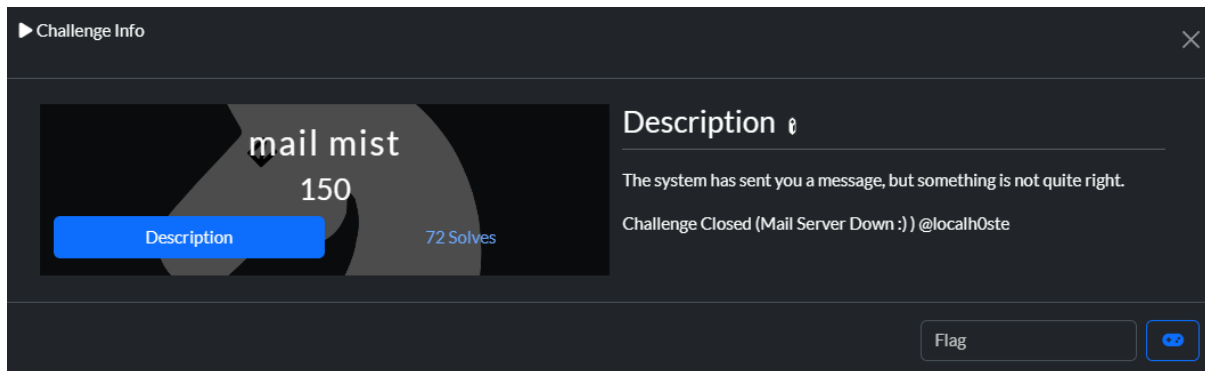


Challenge Name: mail mist - 150 (MISC)



The chall was down. So, I was unable to perform it and share screenshots but I've saved the flags.

Steps (what I did)

1. **Open the challenge page** (the form shown in the screenshot).
The page asks: **Enter your email to receive** and shows a small input + “Send Mail” button.
2. **Create a temporary mailbox / webhook inbox**
 - I used a temporary webhook/email capture.
 - These services give you a unique address **like** `1f9718a8-2fb1-4345-a6d2-a57e90eefe82@emailhook.site` (shown in the page output).
3. **Submit the generated email to the form**
 - Paste the generated address into the form and click **Send Mail**.
 - The page responded: “Mail sent successfully to 1f9718a8-2fb1-4345-a6d2-a57e90eefe82@emailhook.site! Check your email Box”.
4. **Open the webhook / temp inbox**
 - Switch back to the webhook.site page for your inbox.
 - The incoming message from the challenge will appear in the inbox list.
5. **Inspect the captured email / request**
 - The email body (**or the raw webhook payload**) contained the flag that revealed the flag.
 - Copy the flag value from the email.
6. **(Optional) Automate / inspect headers**
 - You can download the raw message or view headers to see the sender, subject, and any hidden data.
 - This can be useful to confirm whether the server leaked anything else (**tokens, internal URLs**).

Why this worked

The challenge expected you to realize the application will send the flag-containing message to whatever email you give it. By using a temporary mailbox that exposes the full incoming message (**including content and headers**), you can trivially read the flag without needing access to the target mail server.

Final

Simple and neat — submit a disposable mailbox, capture the incoming message, and read the flag in plain sight. Nice and quick!

Flag: `SPL{d4c0ba9f5603cf4b0a4db9be6010820a}`

`~By: 1nfln1ty`